Practical Report - File Transfer over TCP/IP in CLI

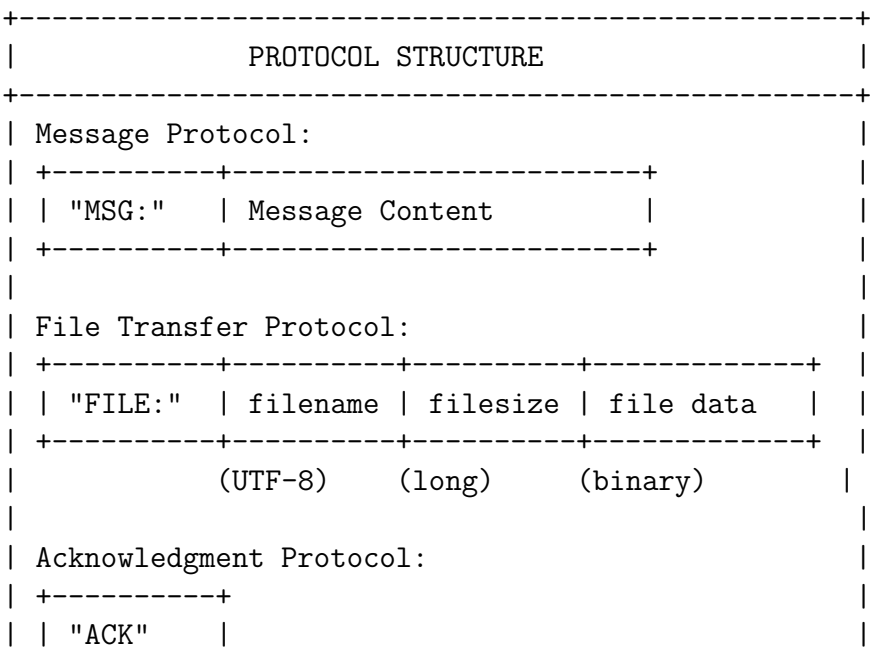
Nguyen Pham Truong An - 23BI14004

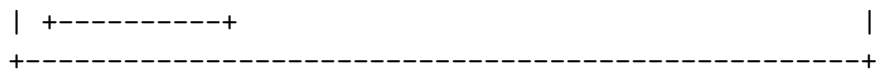
December 4, 2025

1 Protocol Design

1.1 Message Protocol Framework

The system implements a framed protocol to distinguish between chat messages and file transfers, ensuring stream integrity and preventing data corruption.





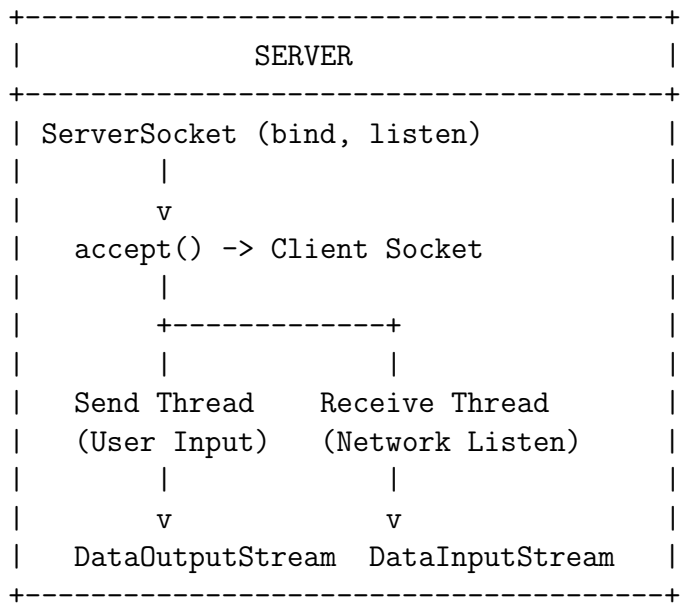
Protocol Types:

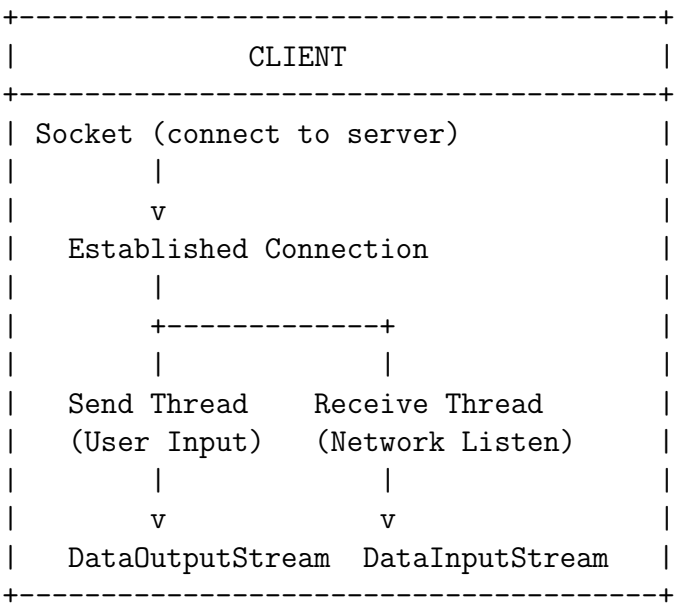
- `MSG:` - Text message with content prefixed by `MSG:`
- `FILE:` - File transfer initiation marker
- `ACK` - Acknowledgment for successful file receipt
- `/quit` - Graceful disconnection command
- `/send <filepath>` - User command to initiate file transfer

2 System Architecture

2.1 Client-Server Organization

The system implements a bidirectional, one-to-one TCP connection with multi-threaded communication.





Key Components:

- **Server**: Listens on port 8080, accepts one client connection, handles bidirectional communication
- **Client**: Connects to server via hostname resolution (`gethostbyname`), establishes TCP socket
- **Threading Model**: Each endpoint runs two threads - one for sending (user input) and one for receiving (network data)
- **Socket Operations**: Implements all core operations - socket(), bind(), listen(), accept(), connect(), send(), recv()

3 File Transfer Implementation

3.1 Transfer Flow with Atomic Operations

The file transfer implements a three-phase protocol ensuring atomicity:

```
PHASE 1: TRANSMISSION
  Sender                    Receiver
```

```
    |                           |
    |---- "FILE:" ------------->|
    |---- filename ------------>| (UTF-8 string)
    |---- filesize ------------>| (long, 8 bytes)
    |---- file data ----------->| (binary stream)
    |                           |
    |                           | Write to "recv_filename"

PHASE 2: ACKNOWLEDGMENT
    |                           |
    |<------- "ACK" ------------| File successfully received

PHASE 3: FINALIZATION
    | Delete original file      |
    |                           | Rename: recv_filename -> filename
```

3.2 Core File Transfer Code

Listing 1: Client/Server File Sending

```
1  private void sendFile(String path) {
2      try {
3          File file = new File(path);
4          if (!file.exists()) {
5              System.out.println("File not found: " + path
                   );
6              return;
7          }
8
9          // Send file transfer protocol header
10         dos.writeUTF("FILE:");
11         dos.writeUTF(file.getName());
12         dos.writeLong(file.length());
13
14         // Stream file data in 4KB chunks
15         FileInputStream fis = new FileInputStream(file);
16         byte[] buffer = new byte[BUFFER_SIZE];
17         int read;
18         while ((read = fis.read(buffer)) != -1) {
```

```
            dos.write(buffer, 0, read);
        }
        dos.flush();
        fis.close();

        // Wait for acknowledgment
        String ack = dis.readUTF();
        if (ack.equals("ACK")) {
            if (file.delete()) {
                System.out.println("Transferred: " +
                    file.getName());
            } else {
                System.out.println("Sent but failed to
                    delete: "
                    + file.getName());
            }
        }
    } catch (IOException e) {
        System.err.println("Send failed: " + e.
            getMessage());
    }
}
```

Listing 2: Client/Server File Receiving

```
private void receiveFile() {
    try {
        // Read file metadata
        String filename = dis.readUTF();
        long size = dis.readLong();

        // Write to temporary file with recv_ prefix
        FileOutputStream fos = new FileOutputStream("
            recv_" + filename);
        byte[] buffer = new byte[BUFFER_SIZE];
        long remaining = size;
        int read;

        // Read exact file size to prevent over-reading
        while (remaining > 0 &&
               (read = dis.read(buffer, 0,
```

```
16                      (int)Math.min(buffer.length,
                           remaining))) != -1) {
17              fos.write(buffer, 0, read);
18              remaining -= read;
19          }
20          fos.close();
21
22          // Send acknowledgment
23          dos.writeUTF("ACK");
24          dos.flush();
25
26          // Atomic rename: recv_filename -> filename
27          File tempFile = new File("recv_" + filename);
28          File finalFile = new File(filename);
29          if (tempFile.renameTo(finalFile)) {
30              System.out.println("Received: " + filename);
31          } else {
32              System.out.println("Received but rename
                   failed: recv_"
33                   + filename);
34          }
35      } catch (IOException e) {
36          System.err.println("Receive failed: " + e.
               getMessage());
37      }
38  }
```

3.3 Socket Operations Mapping

4 Key Implementation Features

4.1 Atomic File Transfer

- **Temporary Prefix**: Files written with `recv_` prefix during transfer
- **Acknowledgment Protocol**: Sender only deletes after receiving `ACK`
- **Atomic Rename**: Receiver renames file only after successful write and ACK sent

Socket Operation	Java Implementation
`socket()`	`new Socket()`, `new ServerSocket()`
`setsockopt()`	`setReuseAddress()`, `setSoTimeout()`
`bind()`	`serverSocket.bind(InetSocketAddress)`
`gethostbyname()`	`InetAddress.getByName(host)`
`listen()`	Implicit in `ServerSocket` creation
`connect()`	`socket.connect(InetSocketAddress)`
`accept()`	`serverSocket.accept()`
`send()`	`DataOutputStream.write()`, `writeUTF()`
`recv()`	`DataInputStream.read()`, `readUTF()`

Table 1: Socket API to Java Mapping

- **Safety**: Prevents partial files and ensures transaction-like behavior

4.2 Protocol Framing

- **Clear Separation**: `MSG:` prefix distinguishes text from binary data
- **Size Tracking**: File size header prevents stream corruption
- **UTF-8 Encoding**: Filename transmitted as UTF-8 for international support
- **Binary Integrity**: Raw byte transfer after headers maintains file integrity

4.3 Error Handling

- **File Validation**: Checks file existence before transmission
- **Connection Resilience**: Handles network failures gracefully
- **Resource Management**: Proper stream closure in finally blocks
- **Timeout Configuration**: 30-second socket timeout prevents hanging

4.4 Concurrency Model

- **Dual Threading**: Separate threads for sending and receiving
- **Volatile Flag**: `volatile boolean running` for thread-safe shutdown
- **Non-blocking Input**: User can type while receiving messages/files
- **Thread Coordination**: Clean shutdown via `/quit` command

5 Usage Examples

5.1 Compilation and Execution

```
# Compile
javac Server.java Client.java

# Run Server (Terminal 1)
java Server 8080

# Run Client (Terminal 2)
java Client 127.0.0.1 8080
```

5.2 Sample Session

```
Server Terminal:
$ java Server 8080
Server listening on port 8080
Client connected: 127.0.0.1
Hello from server!
Client: Hi server!
/send document.pdf
Transferred: document.pdf
Received: report.txt

Client Terminal:
$ java Client 127.0.0.1 8080
Connecting to 127.0.0.1:8080
Connected
```

```
Server: Hello from server!
Hi server!
Received: document.pdf
/send report.txt
Transferred: report.txt
```

6 Conclusion

This implementation demonstrates a complete TCP/IP file transfer system with the following achievements:

- Full implementation of core socket operations
- Atomic file transfer with acknowledgment protocol
- Bidirectional communication with chat capability
- Thread-safe concurrent send/receive operations
- Robust error handling and resource management
- Clean protocol framing preventing stream corruption

The system successfully demonstrates TCP/IP concepts while providing practical file transfer functionality in a command-line environment.